

CX 4220/CSE 6220 High Performance Computing
HW 3 Solutions

Algorithm 1 P-REDUCE

1. (a) **function** P-REDUCE(x, i, j)
 if $i == j$ **then**
 return $x[i]$
 else
 $mid = \lfloor (i + j)/2 \rfloor$
 $left_result = \text{spawn P-REDUCE}(x, i, mid)$
 $right_result = \text{P-REDUCE}(x, mid + 1, j)$
 sync
 return $left_result \otimes right_result$
 end if
end function

(b) **Work:** The total work for P-SCAN-1 is given by:

$$W(n) = \sum_{l=1}^n \Theta(l) = \Theta(n^2)$$

Span: The parallel for assignment will have a span of $\Theta(\log n)$ due to binary forking. The span for each call to P-REDUCE is $\Theta(\log l)$, and the maximum span is $\Theta(\log n)$. Hence to total span $S(n)$:

$$S(n) = \Theta(\log n)$$

Parallelism: The parallelism of the algorithm is the ratio of work to span:

$$\text{Parallelism} = \frac{W(n)}{S(n)} = \frac{\Theta(n^2)}{\Theta(\log n)} = \Theta\left(\frac{n^2}{\log n}\right)$$

(c) **Correctness:**

- **Base Case:** When $i = j$, $y[i] = x[i]$, which is correct.
- **Recursive Case:** First, we observe that the recursive calls operate on disjoint segments, preventing race conditions. Next, one can verify by induction that the serialization of the routine is correct, because
 - the left recursive call calculates the prefix sum up to the pivot, and
 - the right recursive call calculates a prefix sum from the pivot to the upper bound, and,
 - the parent call correctly combines the result of the left recursive call with each element from the right recursive call.

Therefore, the parallel routine is correct as well, since its semantics is identical to that of its serialization, provided that no race conditions are present.

Work:

Work follows the recursive relation: $T_1(n) = 2T_1(n/2) + \Theta(n)$

By Master formula, the total work for P-SCAN-2 is:

$$W(n) = T_1(n) = \Theta(n \log n)$$

Span:

Span follows the recursive relation: $T_\infty(n) = T_\infty(n/2) + \Theta(\log n)$

The span of the algorithm is:

$$S(n) = T_\infty(n) = \Theta(\log^2(n))$$

Parallelism: The parallelism of the algorithm is:

$$\text{Parallelism} = \frac{W(n)}{S(n)} = \frac{\Theta(n \log n)}{\Theta(\log^2(n))} = \Theta\left(\frac{n}{\log n}\right)$$

(d) **Missing Expressions**

P-SCAN-UP (Line 8):

return $t[k] \otimes \text{right}$

P-SCAN-DOWN (Lines 5 and 6):

Line 5: $P\text{-SCAN-DOWN}(v, x, t, y, i, k)$

Line 6: $P\text{-SCAN-DOWN}(v \otimes t[k], x, t, y, k + 1, j)$

Correctness Argument

The two-pass approach in **P-SCAN-3** ensures correctness by:

- **Base Case:** When $i = j$, both **P-SCAN-UP** and **P-SCAN-DOWN** directly return or update with $x[i]$, which trivially satisfies the condition.
- **Upward Pass:** Recursively computes partial results by dividing the array and combining them using $t[k] \otimes \text{right}$.
- **Downward Pass:** Propagates these results downwards. The left subtree receives the current accumulated value v , while the right subtree receives $v \otimes t[k]$, ensuring that all elements up to $i - 1$ are included in the computation for each element $y[i]$.

(e) **Work:** The total work for both **P-SCAN-UP** and **P-SCAN-DOWN** is:

$$W(n) = \Theta(n)$$

Span: The span for both phases is:

$$S(n) = \Theta(\log n)$$

Parallelism: The parallelism of the algorithm is:

$$\text{Parallelism} = \frac{W(n)}{S(n)} = \frac{\Theta(n)}{\Theta(\log n)} = \Theta\left(\frac{n}{\log n}\right)$$

(f) To eliminate the temporary array t :

- Modify **P-SCAN-UP** to store intermediate results directly in sections of the output array y .
- **P-SCAN-DOWN** do not need an extra parameter t and, modify **P-SCAN-DOWN** recursive call, to update v as $v \otimes y[k]$

In the base case, y is over-written as $y[i] = v \otimes x[i]$. So since you are not reducing on y , you can use y to modify v instead of extra variable t .

Algorithm 2 P-SCAN-3

function P-SCAN-3(x, n) let $y[1 : n]$ be new array $y[1] = x[1]$ **if** $n > 1$ **then** P-SCAN-UP($x, y, 2, n$) P-SCAN-DOWN($x[1], x, y, 2, n$) **end if** **return** y **end function****function** P-SCAN-UP(x, y, i, j) **if** $i == j$ **then** **return** $x[i]$ **else** $k = \lfloor (i + j)/2 \rfloor$ $y[k] = \text{spawn P-SCAN-UP}(x, y, i, k)$ $right = \text{P-SCAN-UP}(x, y, k + 1, j)$

sync

return $y[k] \otimes right$ **end if****end function****function** P-SCAN-DOWN(v, x, y, i, j) **if** $i == j$ **then** $y[i] = v \otimes x[i]$ **else** $k = \lfloor (i + j)/2 \rfloor$ **spawn** P-SCAN-DOWN(v, x, y, i, k) P-SCAN-DOWN($v \otimes y[k], x, y, k + 1, j$)

sync

end if**end function**

(g) **P-SCAN-4 Algorithm**

Algorithm 3 P-SCAN-4

```
function P-SCAN-4( $x, n$ )
  if  $n > 1$  then
    P-SCAN-UP( $x, 2, n$ )
    P-SCAN-DOWN( $x[1], x, 2, n$ )
  end if
  return  $x$ 
end function

function P-SCAN-UP( $x, i, j$ )
  if  $i == j$  then
    return  $x[i]$ 
  else
     $k = \lfloor (i + j)/2 \rfloor$ 
     $x[k] = \text{spawn P-SCAN-UP}(x, i, k)$ 
     $right = \text{P-SCAN-UP}(x, k + 1, j)$ 
    sync
    return  $x[k] \otimes right$ 
  end if
end function

function P-SCAN-DOWN( $v, x, i, j$ )
  if  $i == j$  then
     $x[i] = v \otimes x[i]$ 
  else
     $k = \lfloor (i + j)/2 \rfloor$ 
    spawn P-SCAN-DOWN( $v, x, i, k$ )
    P-SCAN-DOWN( $v \otimes x[k], x, k + 1, j$ )
    sync
  end if
end function
```

(h) **Parallel Algorithm for Well-Formed Parentheses**

- i. **Convert** the string of parentheses into an array where each ($\rightarrow +1$ and each) $\rightarrow -1$.
- ii. **Perform a parallel prefix sum** on this array.
- iii. **Check** the results:
 - Ensure all prefix sums are non-negative.

- Verify that the final sum is zero.

If both conditions are met, the string is well-formed; otherwise, it is not.

2. To determine if the parallel prefix algorithm can be used, we need to check if the given binary operations are associative.

1. $a \oplus b = 2a + b$

To check associativity, we need to verify if $(a \oplus b) \oplus c = a \oplus (b \oplus c)$.

Left side: $(a \oplus b) \oplus c = (2a + b) \oplus c = 2(2a + b) + c = 4a + 2b + c$

Right side: $a \oplus (b \oplus c) = a \oplus (2b + c) = 2a + (2b + c) = 2a + 2b + c$

Since $4a + 2b + c \neq 2a + 2b + c$, the operation is not associative. Therefore, the parallel prefix algorithm cannot be used with this operation.

2. $a \oplus b = \sqrt{a^2 + b^2}$

To check associativity, we need to verify if $(a \oplus b) \oplus c = a \oplus (b \oplus c)$.

Left side: $(a \oplus b) \oplus c = (\sqrt{a^2 + b^2}) \oplus c = \sqrt{(\sqrt{a^2 + b^2})^2 + c^2} = \sqrt{a^2 + b^2 + c^2}$

Right side: $a \oplus (b \oplus c) = a \oplus (\sqrt{b^2 + c^2}) = \sqrt{a^2 + (\sqrt{b^2 + c^2})^2} = \sqrt{a^2 + b^2 + c^2}$

Since both sides are equal, the operation is associative. Therefore, the parallel prefix algorithm can be used with this operation.

3. Define $y_i = \frac{s_i}{2^i} = \min\left(\frac{2s_{i-1}}{2^i}, \frac{x_i}{2^i}\right) = \min\left(\frac{s_{i-1}}{2^{i-1}}, \frac{x_i}{2^i}\right) = \min(y_{i-1}, \frac{x_i}{2^i})$.

Compute $y_0, y_1, \dots, y_{n-1}$ by computing parallel prefix on $x_0, \frac{x_1}{2}, \frac{x_2}{2^2}, \dots, \frac{x_{n-1}}{2^{n-1}}$ with min as the operator (min operator is binary associative).

For each $0 \leq i < n$: $s_i = y_i \times 2^i$.

The run-time is dominated by parallel prefix.

Computation time: $O\left(\frac{n}{p} + \log p\right)$

Communication time: $O((\tau + \mu) \log p)$

4. The operator is given to be binary associative. So a Parallel Prefix Sum operation is possible using this operator.

(a) Perform Parallel Exclusive Prefix Sum using the given binary associative operator (PP)

(b) Perform Parallel Exclusive Suffix Sum using the given binary associative operator (PS)

(c) Compute $\text{result}[i] = \text{PP}[i] \oplus \text{PS}[i]$

Computation Time = $\Theta\left(\frac{n}{p} + \log p\right)$

Communication Time = $\Theta((\tau + \mu) \log p)$

Space = $\Theta(n)$